



Test Driven Development

Práctico TEST

1. Precondiciones
2. Pasos del caso de prueba
3. Resultados (assert, assert all)
4. Mensaje de final ya sea éxito o fracaso

▼ Desarrollo Dirigido por Pruebas (TDD)

TDD es un **método** para el desarrollo de *testing*. Su objetivo principal es asegurar la **calidad del código** y mitigar problemas en todo el ciclo de desarrollo, logrando un proceso mental riguroso que facilita la comprensión del dominio y la anticipación de todas las pruebas requeridas.

El TDD impulsa propiedades de diseño deseables, como la **alta cohesión** y el **bajo acoplamiento**, manteniendo la menor cantidad de dependencias posible.

Enfoque Proactivo vs. Reactivo

El TDD representa un cambio de paradigma fundamental respecto al modelo tradicional:

- En el desarrollo **tradicional**, el control de calidad es **reactivo**, ya que se realiza al final del ciclo.
- Con **TDD**, el control de calidad se aborda de manera **proactiva**, comenzando antes de la implementación del código. Esta anticipación ayuda a clarificar los requisitos, ya que es necesario definir *qué* debe hacer el código antes de escribir *cómo* lo hará.
- Aunque el método surge en el contexto de los **procesos ágiles**, su aplicación es válida en cualquier proceso de desarrollo **iterativo y definido**.

▼ El Ciclo de TDD: Rojo, Verde, Refactorizar

El desarrollo se guía por un ciclo constante de tres fases, un mecanismo que garantiza que cada incremento de funcionalidad se valide de inmediato:

1. Rojo : Fallar Primero (Especificación)

En esta etapa se **diseña la prueba unitaria** para la funcionalidad deseada sin implementar aún el código de producción. La prueba debe **fallar** intencionalmente (Rojo) porque el código necesario está ausente.

- **Estructura y Ejemplos:** Las pruebas se estructuran a través de elementos formales:
 - **Precondiciones:** La definición del estado inicial (ej., "Se cuenta con un usuario logueado con permisos de administrador").
 - **Pasos del Caso de Prueba:** La secuencia de acciones a ejecutar.
 - **Nomenclatura:** Los métodos deben comenzar con `test` y usar *snake case* (ej., `test_pedir_taxi`).

2. Verde : Implementación Mínima

Se escribe el **código de producción mínimo** y más simple que logre que la prueba recién fallida pase a ser exitosa.

- **Principio de la Simplicidad:** El foco está solo en pasar la prueba. Por ejemplo, si una función `sumar(2, 2)` debe devolver `4`, el código mínimo inicial podría ser `return 4`. La implementación correcta (`return a + b`) solo se escribe cuando una segunda prueba (`sumar(5, 3)`) obliga a generalizar la lógica.

3. Refactorizar : Optimizar el Diseño

Con las pruebas en verde, se **refactoriza** el código para mejorar su estructura interna y legibilidad, pero **sin alterar el comportamiento** ya validado.

- **Impacto en el Diseño:** Esta fase es crucial para mantener la calidad, ya que se eliminan duplicidades y se optimiza la estructura, asegurando la **alta cohesión** y el **bajo acoplamiento**.

▼ Práctica Avanzada y Validación del Sistema

Niveles de Testing

Aunque el enfoque inicial es la **prueba unitaria**, el proceso de validación es multinivel:

- **Pruebas Unitarias:** Se orientan a verificar la unidad de código.
- **Pruebas de Escenario/Integración:** Prueban la funcionalidad completa de una *user story* o verifican que los componentes trabajen juntos, respectivamente.

Estrategias de Implementación de Pruebas

La construcción de la prueba es inherentemente **iterativa**, y se utilizan estrategias avanzadas para mantener la velocidad y el aislamiento:

- **Dobles de Prueba (Mocks):** Es **válido** y necesario utilizar *mocks* o simulación (*truchear* la prueba) para aislar la unidad de código y probar todos los casos posibles. Esto se hace cuando el código bajo prueba depende de un componente lento o externo (ej., una base de datos o un servicio web), permitiendo que la prueba unitaria siga siendo **rápida** y **autónoma**.
 - *Ejemplo de Mocking:* Para probar una clase que registra un pedido en una base de datos, se usa un *mock* del objeto de la base de datos para simular la operación de guardado, sin acceder realmente al disco o a la red.
- **Resultados y Finalización:** La prueba detalla los **resultados esperados** mediante sentencias **assert** o **assert all**, y debe indicar claramente un mensaje final de **éxito o fracaso**.

Cobertura y Límites

- **Aseguramiento de la Cobertura:** El proceso de TDD asegura la **cobertura de prueba**, garantizando que los *testspasen* por todas las líneas del código.
- **Riesgos Residuales:** Es crucial entender que, a pesar de que las pruebas unitarias y de integración pasen y se logre una alta cobertura, **el código aún puede contener defectos**. Por lo tanto, las

actividades de *testing* y el control de calidad externos siguen siendo necesarios para validar escenarios que las pruebas automatizadas no pueden capturar (ej., usabilidad o rendimiento).